



Bus Factor HQ

Simulador arcade de emergencias organizacionales

"¿Qué pasa si mañana tu CTO no llega? Nosotros ya lo simulamos... y tu empresa sobrevivió."

1. ¿Qué es?

Bus Factor HQ es un copiloto de continuidad organizacional con interfaz de videojuego arcade. Se conecta a las herramientas donde vive el conocimiento real de una empresa — Slack, GitHub, Notion, Linear y las grabaciones de Google Meet en Drive— y construye un **mapa vivo de "quién sabe qué"**: tareas, responsables, accesos, dependencias y conocimiento tácito (esas reglas no escritas como *"al jefe solo le gusta viajar en LATAM"*).

Con ese mapa, el sistema permite:

1. **Simular emergencias**: seleccionas un escenario —renuncia un miembro clave, roban el computador del CTO, cae GitHub— y ves en tiempo real, sobre una oficina pixel art, qué tareas quedan huérfanas, qué se rompe en cascada y cuánto tardaría el equipo en recuperarse.
2. **Prevenir antes de simular**: cada viernes genera un **digest de descentralización** con misiones concretas y accionables tipo *"Quest: David → compartir acceso del servidor con Samuel"* o *"Quest: Andrés → documentar el flujo del PR pendiente"*. Completar quests sube el puntaje de resiliencia del equipo.

Es un producto **multiusuario**: cada miembro del equipo tiene su cuenta (usuario y contraseña), personaliza su avatar y pertenece a una oficina virtual. Cada quien conecta sus propias fuentes —su Slack y su carpeta de Drive con las grabaciones de sus reuniones— y su personaje cobra vida dentro de la oficina.

El nombre juega con el concepto de **bus factor**: el número de personas que pueden "ser atropelladas por un bus" antes de que un proyecto colapse. Nuestro objetivo: que ese número nunca sea 1.

2. El pitch (guion de 3 minutos)

Gancho (30 seg):

“Levanten la mano si en su equipo hay algo que *solo una persona* sabe hacer. [pausa] Exacto. Todos tenemos un bus factor de 1 en algo. Y el día que esa persona falte — renuncia, se enferma, le roban el equipo— la empresa entra en modo pánico: empalmes de semanas, conocimiento perdido, clientes esperando.”

Problema (30 seg):

“El conocimiento crítico de las empresas no vive en la documentación. Vive en threads de Slack, en reuniones de Meet que nadie vuelve a ver, en la cabeza de la gente. Documentar manualmente no funciona: nadie tiene tiempo y la documentación nace desactualizada.”

Solución + demo (90 seg):

“Bus Factor HQ lee lo que su equipo ya produce —sin trabajo extra— y construye el mapa de conocimiento automáticamente. [DEMO EN VIVO: mostrar la oficina pixel art con los 9 personajes, uno brillando en rojo] Esta es nuestra oficina real. Ana está en rojo: es la única que sabe comprar los vuelos del jefe y administrar el CRM. Simulemos que mañana no llega. [clic → Ana desaparece → alertas en cascada] El sistema identifica 7 tareas huérfanas y genera en 10 segundos el documento de empalme: qué hacía, cómo, con quién, e incluso que al jefe solo le gusta LATAM. [mostrar digest] Y para que esto nunca pase, cada viernes reparte quests de descentralización. Prevenir es un juego.”

Cierre (30 seg):

“Las empresas gastan millones en backups de sus datos, pero cero en backups de su conocimiento. Bus Factor HQ es el plan de contingencia que se construye solo. Porque la pregunta no es *si* va a pasar una emergencia, sino *cuándo*.”

3. ¿Por qué genera impacto?

Ángulo	Argumento
Dolor universal	Todo jurado, mentor o empresa ha vivido la salida traumática de alguien clave. No hay que explicar el problema: lo reconocen al instante.
Costo real	Reemplazar a un empleado cuesta entre 6 y 9 meses de su salario, y gran parte es pérdida de conocimiento y tiempo de empalme. En equipos pequeños (como el nuestro, de 9), un bus factor de 1 puede frenar la operación completa.

Cero fricción de adopción	No pide que nadie documente nada ni cambie su forma de trabajar. Se alimenta de lo que el equipo ya produce.
Prevención medible	El puntaje de resiliencia convierte algo intangible ("¿qué tan preparados estamos?") en un número que mejora semana a semana.
Aplicable más allá de tech	Cualquier organización con Slack/Meet/Drive tiene este problema: agencias, colegios, hospitales, gobierno.

4. Diferenciador

Los productos existentes atacan pedazos del problema:

- **Wikis y documentación (Notion, Confluence):** dependen de que humanos documenten. Nadie lo hace.
- **Herramientas de offboarding (HR):** actúan cuando ya es tarde, con checklists genéricos.
- **Análisis de bus factor en código (herramientas de Git):** solo miran repositorios, ignoran el 80% del conocimiento que no es código.

Nuestro diferenciador es triple:

1. **Conocimiento tácito, no solo tareas.** Capturamos las reglas no escritas (preferencias, contexto, "así se hace aquí") desde conversaciones y reuniones reales. Nadie más está leyendo las transcripciones de Meet para esto.
2. **Simulación proactiva.** No esperamos la emergencia: la ensayamos. Es un "simulacro de incendio" pero para el conocimiento organizacional.
3. **Gamificación real.** La prevención es aburrida y por eso nadie la hace. Al convertirla en quests semanales con puntaje visible, el equipo *quiere* descentralizar. La interfaz arcade no es decoración: es la estrategia de adopción.

5. ¿Qué hará que ganemos?

1. **Demo con historia, no con features.** Vamos a simular en vivo la desaparición de un miembro real del equipo y generar su documento de empalme en segundos. Es visual, emocional y memorable.

2. **La interfaz arcade rompe el molde.** Entre veinte demos de dashboards y chatbots, una oficina pixel art donde un personaje desaparece y todo entra en alerta se queda en la memoria del jurado.
 3. **Uso inteligente de IA, no IA por moda.** RAG + grafo de conocimiento + extracción de entidades: arquitectura defendible técnicamente, sin humo de “entrenamos nuestro propio modelo”.
 4. **Problema de negocio serio con empaque divertido.** El jurado técnico valora la arquitectura; el jurado de negocio valora el dolor real y el mercado; todos disfrutan el juego.
 5. **Alcance honesto y funcional.** Preferimos 1 historia completa funcionando (ingesta real → riesgo detectado → simulación → playbook generado) que 5 integraciones a medias.
-

6. Experiencia multiusuario y diseño audiovisual

Cuentas, oficina y avatares

- Cada miembro se registra con **usuario y contraseña** (Supabase Auth) y pertenece a una **Oficina** (organización). Sin oficina no hay personaje: la pertenencia es el núcleo del modelo.
- Al entrar por primera vez, el usuario **crea su avatar por capas**: cuerpo, peinado, ropa y colores (sprites LPC/Kenney). Es un editor barato de construir y un momento demoable delicioso.
- Cada usuario **conecta sus propias fuentes**: su Slack y su carpeta de Drive con sus grabaciones de Meet. Todo `KnowledgeItem` guarda `owner_user_id` y `source_connection_id`, así el mapa de conocimiento sabe de quién viene cada cosa.

⚠ **Recorte de hackathon (importante):** el OAuth individual de Slack y Google (crear apps, pantallas de consentimiento, verificación de Google) consume demasiadas horas. El flujo “Conectar mi Slack / mi Drive” se construye **real para 1-2 cuentas del demo**; los demás usuarios se siembran con datos de prueba. La arquitectura queda diseñada multiusuario desde el día uno; la verificación completa de OAuth es post-hackathon.

La oficina viva

El objetivo audiovisual es que la oficina se sienta **funcionando**, nunca un mapa estático:

- **Personajes con vida propia:** los avatares caminan entre escritorios con pathfinding en grilla, se sientan, teclean, van por café, entran a la sala de juntas. Todo movimiento con

tweening y easing — jamás teletransportes ni saltos bruscos. Esa limpieza de movimiento es la diferencia entre “juguete” y “producto”.

- **Objetos esenciales como entidades interactivas:** el servidor de **GitHub** con luces parpadeando, la sala de juntas con la pantalla de **Meet** donde se reúnen los avatares, **computadores** encendidos, la cafetera, las **luces** del techo.
- **Ambiente:** iluminación cálida con sombras suaves, pantallas que titilan, paneos suaves de cámara, música chiptune tranquila de fondo y efectos de sonido puntuales.
- **Regla de oro del demo:** cada emergencia es primero espectáculo (5-10 segundos de animación) y luego sustancia (el panel serio con tareas huérfanas, impacto y playbook). En ese orden, siempre.

7. Catálogo de escenarios de emergencia

El selector de escenarios es una “consola arcade” dentro de la oficina — muchas opciones visibles transmiten profundidad de producto:

Escenario	Efecto visual	Qué analiza el cerebro
Renuncia / ausencia	El avatar recoge sus cosas y sale por la puerta; su escritorio queda gris	Tareas huérfanas, conocimiento tácito perdido, documento de empalme
Robo del computador del CTO	El PC desaparece, suena alarma, luz roja giratoria	Accesos comprometidos, credenciales a rotar, información expuesta
Caída de GitHub	El servidor echa humo; los devs se levantan confundidos	Procesos bloqueados, respaldos/mirrors, quién puede restaurar
Caída de Meet	La pantalla de juntas queda en estática; la reunión se disuelve	Reuniones críticas afectadas, canales de comunicación alternos
Apagón	Las luces se van; solo brillan las pantallas	Dependencia física, contingencia de trabajo remoto
La oficina se cae (incendio/evacuación)	Screen shake, partículas, todos corren a la salida	Plan de continuidad total, operación 100% remota
Ransomware /	Pantallas en rojo, candados por	Respaldos, accesos offline,

internet caído	todas partes	protocolo de respuesta
----------------	--------------	------------------------

Para el demo en vivo se ejecutan 1-2 escenarios completos; el resto queda visible en la consola como opciones (y como respuesta lista para el "¿qué más puede simular?" del jurado).

8. Stack tecnológico

Capa	Tecnología	Por qué
LLM / IA	Claude API (extracción de entidades, generación de playbooks y quests)	No entrenamos nada propio: RAG + prompts estructurados. Rápido, demostrable, defendible.
Embeddings + vector store	Supabase con pgvector (o Chroma local si hay problemas)	Supabase da BD relacional + vectores + auth en un solo servicio gratuito.
Backend / API	Python + FastAPI	El equipo ya domina Python; FastAPI es rápido de montar y auto-documenta la API (clave para integrar sin fricciones).
Ingesta	Slack API, GitHub API (Octokit/REST), Google Drive API (transcripciones de Meet como Docs)	APIs oficiales, bien documentadas, con tokens de prueba fáciles de generar.
Frontend juego	React + Phaser.js	Phaser es el estándar para 2D arcade en web y convive bien con React.
Frontend dashboard	React + Tailwind CSS	Digest de viernes, quests, puntaje de resiliencia y detalle del playbook.
Auth multiusuario	Supabase Auth	Usuario/contraseña, sesiones y pertenencia a oficinas sin construir nada desde cero.
Assets visuales	Kenney.nl + LPC (sprites pixel art gratuitos, CC0/CC-BY)	Cero tiempo dibujando; LPC trae personajes por capas, ideal para el editor de avatar.
Repositorio y	GitHub (monorepo) + Vercel (front) + Railway/Render (back)	Deploy en minutos; si falla, el demo

despliegue		corre en localhost sin vergüenza.
------------	--	-----------------------------------

Modelo de datos central (el contrato de todo el equipo)

```
Office      { id, nombre }
User        { id, email, office_id, avatar_config }
Connection  { id, user_id, tipo: slack|drive, estado }
Person      { id, user_id, nombre, rol, sprite }
KnowledgeItem { id, tipo: tarea|acceso|regla_tacita|proceso,
                descripcion, dueño_principal, respaldos[],
                fuente: slack|github|meet|notion, evidencia }
RiskScore   { person_id, score 0-100, items_criticos[] }
Scenario    { id, tipo: persona|infra|fisica, objetivo, descripcion }
SimulationResult { scenario_id, items_huerfanos[], impacto,
                playbook_md }
Quest       { id, asignado_a, accion, item_relacionado,
                estado: pendiente|completada, puntos }
```

9. Recursos necesarios (conseguir ANTES de la hackathon)

- **API key de Claude** (Anthropic) con crédito suficiente
- **Workspace de Slack de prueba** con historial realista (podemos sembrar conversaciones del equipo real, anonimizadas si hace falta)
- **Repo de GitHub de prueba** con commits y PRs de varios autores
- **3-5 transcripciones de Meet** exportadas de la carpeta de Drive (verificar que las transcripciones automáticas estén activadas; si solo hay video, transcribir antes con Whisper — no durante la hackathon)
- **Proyecto de Supabase** creado, con pgvector habilitado
- **Tokens/OAuth apps** de Slack, GitHub y Google Cloud (Drive API) ya configurados y probados
- **Assets de Kenney** descargados (pack de oficina/interiores + personajes)
- **Sprites LPC por capas** descargados para el editor de avatar (cuerpo, peinado, ropa)
- **OAuth apps de Slack y Google en modo prueba**, autorizadas solo para las 1-2 cuentas del demo (no intentar verificación completa de Google antes de la hackathon)

- **Música chiptune y efectos de sonido** libres de derechos (alarma, humo, apagón) descargados
- **Monorepo creado** con estructura de carpetas, linters y README de contratos
- Datos de demo de respaldo en JSON (por si una API falla en vivo, el show continúa)

10. Plan de implementación — 5 personas, integración sin sufrimiento

Principio rector: contrato primero, integración por citas

Nadie espera a nadie. En la **Hora 0** todo el equipo define juntos los esquemas JSON de la sección 6 (30-45 min, no más). Desde ese momento, **cada persona trabaja contra datos falsos (mocks) que cumplen el contrato**. Si los mocks funcionan, lo real funciona. Las integraciones no se dejan "para el final": tienen citas programadas (hitos) donde dos personas conectan sus piezas y prueban.

Reparto de roles

#	Rol	Responsable de	Entrega (su "contrato")
P1	Ingesta	Conectores de Slack, GitHub y Drive/Meet. Normalizar todo a un formato único.	Endpoint/archivos <code>RawEvent[]</code> en JSON normalizado. Desde la hora 1 publica mocks con datos falsos realistas.
P2	Cerebro IA	Pipeline de extracción con Claude (<code>RawEvent</code> → <code>KnowledgeItem</code>), embeddings, scoring de riesgo, generación de playbooks y quests.	Funciones/endpoints: <code>extraer()</code> , <code>calcular_riesgo()</code> , <code>simular(scenario)</code> , <code>generar_digest()</code> . Consume los mocks de P1 sin esperarlo.
P3	Backend + BD + Auth	FastAPI, Supabase (BD + Auth), modelo de datos multiusuario (<code>Office/User/Connection</code>), orquestación de simulaciones, API REST que consumen	API documentada (Swagger automático de FastAPI) sirviendo datos falsos desde la hora 2 , luego conecta el

		los dos frontends.	cerebro real. Registro/login funcionando temprano.
P4	Frontend Juego	La oficina viva en Phaser + React: tilemap, pathfinding y animaciones ambientales de los personajes, objetos interactivos (GitHub, Meet, PCs, luces), editor de avatar por capas, y los efectos de cada escenario del catálogo (humo, apagón, screen shake, alarma).	La escena del juego consumiendo la API de P3 (primero con datos falsos). Prioridad: movimiento limpio antes que cantidad de escenarios.
P5	Frontend Dashboard + Demo/Pitch	Pantallas de digest de viernes, quests, puntaje, visor del playbook. Además: datos de demo, guion del pitch, deck y QA general.	Dashboard funcional + demo ensayado. Es también el "pegamento" que prueba el flujo completo con ojos de usuario.

Por qué esta división evita el caos: P1→P2, P2→P3 y P3→(P4,P5) forman una cadena donde cada eslabón solo depende del **contrato** del anterior, nunca de su código. Los frontends jamás hablan con Slack ni con Claude directamente: solo con la API de P3. Si una pieza se atrasa, su mock la reemplaza y el demo no muere.

Cronograma de integración (hackathon de ~24-30 horas)

Hito	Cuándo	Quiénes	Qué se prueba
H0 — Contratos	Hora 0-1	Todos	Esquemas JSON aprobados y escritos en el README. Nadie codea antes de esto.
H1 — Mocks vivos	Hora 2-3	Todos	P1 publica RawEvents falsos; P3 sirve una API falsa; P4 y P5 pintan pantallas con esos datos falsos. El demo ya "funciona" de mentiras.
H2 — Primera cita: P1 + P2	Hora 6-8	P1, P2	El cerebro extrae KnowledgeItems desde datos reales de Slack/GitHub. Prueba: ¿salen tareas y dueños coherentes?
H3 — Segunda cita: P2 + P3	Hora 10-12	P2, P3	La API sirve riesgo y simulaciones reales. Prueba: <code>POST /simular</code> devuelve items huérfanos + playbook.
H4 — Tercera cita: P3 + P4 y	Hora	P3, P4,	Los frontends cambian de la API falsa a la real (mismo contrato = cambio de una URL). Prueba

P3 + P5	14-16	P5	visual completa.
H5 — Meet/Drive (si hay tiempo)	Hora 16-18	P1, P2	Se suman las transcripciones de Meet a la ingesta. Si no da tiempo, van como mock en el demo y como roadmap en el pitch.
H6 — Congelamiento	Hora 20	Todos	Se congela el código. Nada de features nuevas. Solo bugs del flujo del demo.
H7 — Ensayo general	Hora 21-23	Todos (dirige P5)	Correr el demo completo 3 veces con cronómetro. Preparar plan B (datos locales) por si el WiFi o una API falla.

Reglas del equipo

1. **El contrato es sagrado.** Cambiarlo requiere avisar a todos; se anota la versión en el README.
2. **Commits pequeños al monorepo, mínimo cada 2 horas.** Nadie guarda trabajo solo en su máquina.
3. **Si algo real no funciona, se demuestra con mock y se sigue.** El jurado evalúa la historia completa, no cada API en vivo.
4. **P5 tiene autoridad de recorte:** si a la hora 16 algo pone en riesgo el demo, se corta sin debate.
5. **Toda decisión técnica dudosa se resuelve con la pregunta:** *"¿esto se ve en el demo de 3 minutos?"* Si no, no se hace.

11. Roadmap post-hackathon (para la pregunta del jurado "¿y después?")

- OAuth individual verificado (Slack + Google) para que cada usuario conecte sus fuentes en autoservicio
- Integración completa de Notion y Linear (hoy en mock/roadmap)
- Detección de conocimiento tácito en tiempo real (bot de Slack que pregunta "¿alguien más sabe esto?")
- Alertas preventivas: "el bus factor de Ana subió a crítico esta semana"
- Modo multiempresa (SaaS) con puntaje de resiliencia comparable por industria

- Simulacros programados automáticos: una vez al mes, el sistema "esconde" a alguien por un día como ejercicio real